
MPaGE

Pareto-Grid-Guided Large Language Models for Fast and High-Quality Heuristics Design in Multi-Objective Combinatorial Optimization

Tóm tắt thuật toán chính

Pareto grid

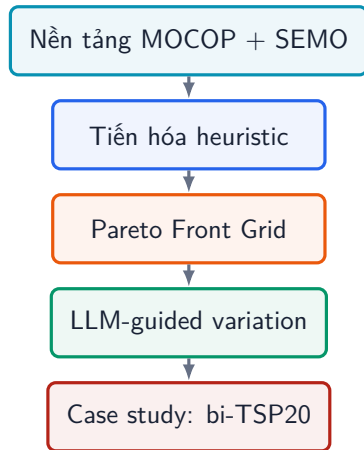
LLM semantic clustering

Code evolution

Minh Hieu Ha, Hung Phan, Tung Duy Doan, Tung Dao, Dao Tran, Huynh Thi Thanh Binh — AAI 2026

Nội dung

- 1 Preliminaries
- 2 Khung MPaGE
- 3 Pareto Front Grid Selection
- 4 LLM-guided Variation
- 5 Case study: evaluator bi-TSP20
- 6 Tổng kết



Preliminaries: MOCOP và Pareto dominance

Một bài toán tối ưu tổ hợp đa mục tiêu có dạng

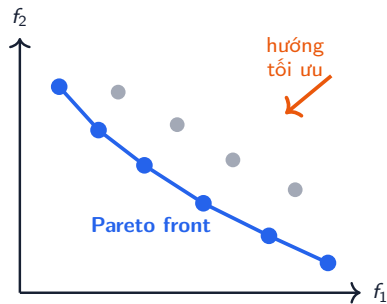
$$\min_{s \in \mathcal{S}} \mathbf{f}(s) = (f_1(s), \dots, f_m(s)),$$

với không gian nghiệm rời rạc $\mathcal{S}$.

Pareto dominance

$$s \prec s' \iff \begin{aligned} f_j(s) &\leq f_j(s') && \forall j, \\ f_j(s) &< f_j(s') && \exists j. \end{aligned}$$

Tập các nghiệm không bị nghiệm nào khác trội tạo thành **Pareto set**; ảnh của nó trong objective space là **Pareto front**.



Hypervolume (HV)

Đo thể tích objective space được Pareto front bao phủ so với một reference point; HV càng lớn, front càng tốt.

Preliminaries: khung SEMO

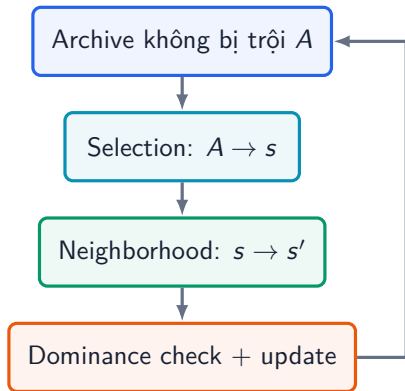
SEMO duy trì archive A gồm các nghiệm không bị trội.

1. Chọn nghiệm cha mẹ $s \in A$ (SEMO gốc: ngẫu nhiên).
2. Chọn ngẫu nhiên một lân cận của s để sinh s' .
3. Tính objective vector $\mathbf{f}(s')$.
4. Nếu s' không bị trội: thêm s' và loại các nghiệm bị s' trội.

Hai quyết định cần thiết kể

Selection: chọn nghiệm nào từ A ?

Neighborhood: biến đổi nghiệm đó thế nào?



Vai trò của MPaGE: dùng LLM tự động thiết kế đồng thời hai quyết định trên dưới dạng một hàm Python.

MPaGE đang tiến hóa cái gì?

Vòng ngoài: heuristic design (LMHD)

Cá thể là một **heuristic hoàn chỉnh** h :

$h = (\text{mô tả tự nhiên, mã Python, } e(h)).$

Mã của h thực hiện cả hai quyết định: chọn nghiệm từ archive và sinh nghiệm lân cận.

MPaGE tiến hóa quần thể heuristic P

Vòng trong: chạy SEMO

Với archive nghiệm hiện tại A , heuristic sinh ứng viên mới:

$$s' = h(A).$$

Nếu s' không bị trội, cập nhật:

$$A \leftarrow \{s'\} \cup \{s \in A : s' \not\prec s\}.$$

Archive nghiệm A

MPaGE không tiến hóa prompt trả lời; nó tiến hóa mã heuristic điều khiển SEMO.

Biểu diễn cá thể và fitness đa mục tiêu

Một heuristic gồm

- ▶ mô tả ý tưởng bằng ngôn ngữ tự nhiên;
- ▶ code Python cho hàm $h(A) \rightarrow s'$;
- ▶ vector fitness đo trên tập training instances.

Khởi tạo

LLM nhận mô tả bài toán, cấu trúc lời giải và chữ ký hàm; sau đó sinh N heuristic ban đầu có logic khác nhau.

Hai objective được tối thiểu hóa

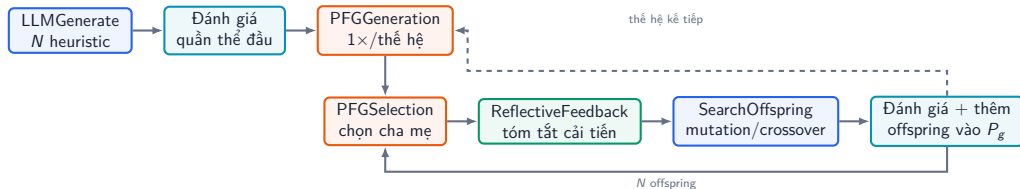
$$\mathbf{e}(h) = (e_1(h), e_2(h))$$

$$e_1(h) = \frac{1}{|\mathcal{I}|} \sum_{l \in \mathcal{I}} -\text{HV}(h(l)),$$

$$e_2(h) = \sum_{l \in \mathcal{I}} T(h(l)).$$

- ▶ e_1 : chất lượng Pareto front qua hypervolume.
- ▶ e_2 : tổng thời gian chạy.

Toàn bộ vòng lặp MPaGE



Mỗi thể hệ

1. dựng PFG một lần từ P_{g-1} ;
2. sinh tuần tự N offspring từ PFG cố định;
3. đánh giá và thêm từng offspring vào P_g .

Kết thúc

- ▶ Lặp trong G thể hệ.
- ▶ Trả về các heuristic không bị trội trong quần thể cuối.

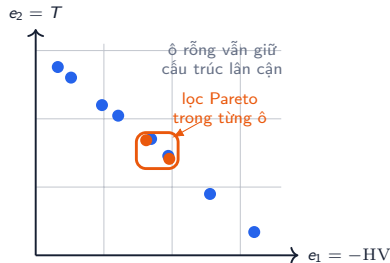
Pareto Front Grid: tạo lưới

Với mỗi objective j , lấy ideal/nadir point z_j^*, z_j^{nad} và margin σ :

$$\delta_j = \frac{z_j^{\text{nad}} - z_j^* + 2\sigma}{K_j},$$

$$g_j(h) = \left\lfloor \frac{e_j(h) - z_j^* + \sigma}{\delta_j} \right\rfloor.$$

- ▶ $\mathbf{g}(h) = (g_1(h), g_2(h))$ là ô chứa heuristic h .
- ▶ Trong mỗi ô, chỉ giữ các cá thể **không bị trội nội bộ**.
- ▶ Hợp các elite theo ô tạo tập E dùng cho selection.



PFG selection: hai nhánh xác suất ϵ

Nếu $u < \epsilon$: nhánh theo lưới

1. Chọn đều một ô đang có elite.
2. Gom elite của ô đó và các ô kề trực tiếp.
3. Gọi **SemClust** để nhóm theo logic code.
4. Tạo offspring bằng mutation hoặc crossover giữa các cụm.

Thực nghiệm: $K_1 = K_2 = 4$, $\epsilon = 0.9$

Nếu $u \geq \epsilon$: nhánh toàn cục

1. Lấy ngẫu nhiên hai elite $h_1, h_2 \in E$.
2. Dùng trực tiếp hai heuristic này làm cha mẹ.
3. Tạo offspring từ hai elite mà không cần semantic clustering cục bộ.

Neighbor: các ô kề theo các trục objective

Semantic clustering và toán tử sinh hậu duệ

LLM-based semantic clustering

- ▶ LLM đọc description và code của từng heuristic.
- ▶ Nhóm theo **hành vi/logic thuật toán**, không dựa vào AST hay embedding cú pháp.
- ▶ Chọn một cụm ngẫu nhiên rồi chọn một heuristic h trong cụm.

Reflection

LLM phân tích heuristic cha mẹ và fitness của chúng, rồi tạo feedback trước khi viết code mới.

Search offspring

$$h_{\text{new}} = \begin{cases} \text{Mutate}(h), & v < \gamma, \\ \text{Cross}(h, h'), & v \geq \gamma, \end{cases}$$

trong đó h' được lấy từ một cụm khác.

- ▶ Mutation: thay đổi một chiến lược hiện có.
- ▶ Crossover: tổng hợp logic bổ sung từ các cha mẹ khác cụm.
- ▶ Reflection dẫn hướng cho cả mutation lẫn crossover.

Thực nghiệm: $\gamma = 0.3$

Thiết lập tái lập trên bi-TSP20

Dữ liệu và thước đo chung

- ▶ Bi-objective TSP, 20 thành phố, 4 instances.
- ▶ Seed dữ liệu: 2025; reference point: [20, 20].
- ▶ Hypervolume dùng cùng `pymoo.indicators.hv.HV`.
- ▶ Mỗi instance: 100 nghiệm khởi tạo + 2.000 lần thử sinh offspring.

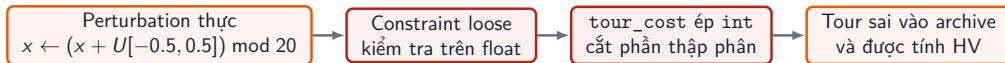
Runtime của SEMO chỉ tính thời gian chạy heuristic; không gồm thời gian LLM sinh heuristic ở tầng thiết kế.

Bốn cấu hình được so sánh

Phương pháp	Vai trò
NSGA-II	baseline hoán vị
MOEA/D	baseline decomposition
SEMO-loose	tái lập evaluator gốc
SEMO-strict	kiểm tra tour hợp lệ

- ▶ NSGA-II/MOEA/D: population 100, 20 thế hệ.
- ▶ Seed thuật toán: 2025 + instance id.
- ▶ Kết quả lưu cả HV, runtime, Pareto front và tours.

Vấn đề trong evaluator bi-TSP20



Tại sao candidate sai vẫn qua?

Constraint gốc chỉ kiểm tra:

- ▶ có đúng 20 phần tử;
- ▶ các giá trị float khác nhau;
- ▶ mọi giá trị thuộc $[0, 20)$.

Ví dụ $5.4 \neq 5.8$ khi kiểm tra, nhưng cả hai đều thành node 5 khi tính cost.

Đây là lỗi implementation/evaluator của case study, không phải lỗi của cơ chế PFG.

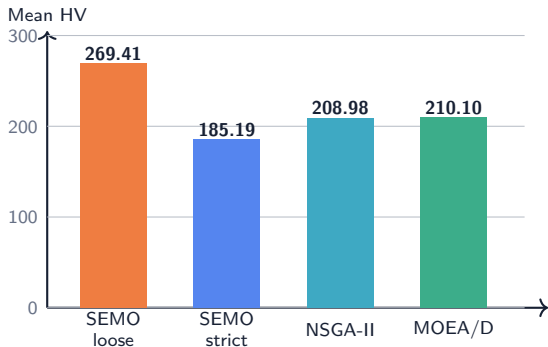
Cách sửa trong SEMO-strict

Dùng cùng một biểu diễn nguyên cho kiểm tra, cost và archive:

$$c = \text{astype}_{\mathbb{Z}}(s'), \quad \text{sort}(c) = [0, 1, \dots, 19].$$

Chỉ khi điều kiện trên đúng mới tính objective và thêm c vào archive.

Tác động định lượng lên hypervolume



Loose → strict

- ▶ Mean HV giảm 84.223349.
- ▶ Tương đương giảm khoảng **31.26%**.
- ▶ Trung bình 1,755/2,000 candidate bị strict loại.
- ▶ Strict chỉ còn trung bình 345.25 objective evaluations.

Kiểm tra Pareto front loose

Front của 4 instances có lần lượt 46, 10, 18 và 28 tours; **không tour nào** trở thành hoán vị 0..19 hợp lệ sau khi ép int.

Diễn giải kết quả và benchmark công bằng

Có thể kết luận gì?

- ▶ HV cao của SEMO-loose phụ thuộc đáng kể vào các tour không hợp lệ.
- ▶ Vì vậy không thể dùng HV loose để kết luận heuristic vượt trội trên bi-TSP20 hợp lệ.
- ▶ SEMO-strict cho thấy tác động của việc sửa constraint, nhưng chưa phải so sánh ngân sách hoàn toàn ngang bằng.

Representation phải nhất quán end-to-end

Chênh lệch ngân sách objective

NSGA-II	2.100
MOEA/D	2.100
SEMO-loose	2.100
SEMO-strict	342–350

Đề xuất đánh giá lại

1. dùng toán tử luân sinh hoán vị hợp lệ;
2. hoặc sinh tiếp đến đủ 2.000 offspring hợp lệ;
3. kiểm tra đúng tập $\{0, \dots, 19\}$ trước cost/archive;
4. báo cáo runtime LLM riêng với runtime heuristic.

Pseudocode rút gọn và thông điệp chính

MPaGE

1. $P_0 \leftarrow \text{LLMGenerate}(\mathcal{L}, \Pi, N)$; đánh giá từng $h \in P_0$.
2. **For** $g = 1, \dots, G$:
 - 2.1 $(\mathcal{G}, E) \leftarrow \text{PFGGeneration}(P_{g-1})$;
 - 2.2 $P_g \leftarrow P_{g-1}$; **repeat** N lần:
 - 2.3 $H_{\text{par}} \leftarrow \text{SelectParents}(\mathcal{G}, E, \epsilon, \gamma)$;
 - 2.4 $\phi \leftarrow \text{ReflectiveFeedback}(\mathcal{L}, H_{\text{par}})$;
 - 2.5 $o \leftarrow \text{SearchOffspring}(\mathcal{L}, \phi)$;
 - 2.6 đánh giá o và $P_g \leftarrow P_g \cup \{o\}$.
3. **return** $P^* = \text{NonDominatedSet}(P_G)$.

Ba thành phần cốt lõi

1. PFG

Duy trì nhiều vùng trade-off HV-runtime.

2. SemClust

Bảo toàn đa dạng về logic code.

3. LLM variation

Reflection + mutation/crossover sinh heuristic mới.

Mỗi run trả về nhiều heuristic

Nguồn: Ha et al., arXiv:2507.20923v3 / AAAI 2026.